

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JASMITHA R	Reg. No.:	192524295
Section / Batch:	AI&DS	Date:	25/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of Dynamic Programming and Greedy algorithms for optimization problems.	Q1

SECTION A – Comparative Analysis

Code of Conduct

I JASMITHA R (192524295)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JASMITHA R

Travelling Salesperson Problem Comparison: Dynamic Programming vs Greedy Nearest Neighbour Heuristic

1. Problem Statement

The Travelling Salesperson Problem (TSP) requires a salesperson to visit every city exactly once, starting and ending at the same city, while minimizing the total travel distance. This assignment considers four cities, labelled A, B, C and D. Two solution strategies are examined and compared: an exact Dynamic Programming solution based on the Held-Karp algorithm, and an approximate Greedy heuristic based on the Nearest Neighbor rule. The comparison covers total tour cost, number of computational steps, correctness guarantees, time and space complexity, and the practical scalability of each approach as the number of cities grows.

2. Distance Matrix

The distance between every pair of cities is given below. The matrix is symmetric, meaning the distance from city X to city Y equals the distance from city Y to city X, and the distance from a city to itself is zero.

City	A	B	C	D
A	0	35	39	40
B	35	0	35	30
C	39	35	0	45
D	40	30	45	0

All distances are assumed to be in kilometers. The salesperson begins the journey at city A, must visit B, C and D exactly once each, and must return to A at the end of the tour.

3. Dynamic Programming Approach (Held-Karp Algorithm)

The Held-Karp algorithm solves TSP exactly using dynamic programming over subsets of cities. It defines a state $C(S, i)$ as the minimum cost of a path that starts at the fixed origin city, visits every city in the set S exactly once, and ends at city i , where i belongs to S . The recurrence builds larger subsets from smaller ones, so every state is computed only once and reused whenever it is needed again, which is the core idea of dynamic programming.

3.1 Algorithm Steps

- Step 1: Fix city A as the starting and ending point of the tour.
- Step 2: Initialize the base cases. For every city i other than A, set $C(\{i\}, i)$ equal to the direct distance from A to i .
- Step 3: For every subset S of the remaining cities of increasing size, and for every city i inside S , compute $C(S, i)$ as the minimum, over every city j in S other than i , of $C(S \text{ minus } i, j)$ plus the distance from j to i .
- Step 4: Repeat Step 3 until subsets covering all cities except A have been processed.
- Step 5: Compute the final answer as the minimum, over every city i , of $C(\text{FullSet}, i)$ plus the distance from i back to A.
- Step 6: Reconstruct the optimal path by tracing back through the stored choices that produced the minimum value at every stage.

3.2 Dynamic Programming Computation Table

Using the labels 1 = B, 2 = C and 3 = D relative to the fixed start city A (labelled 0), the computed subproblem values are shown below.

Subset Size	Subset	End City	C(S, i)
1	{B}	B	35
1	{C}	C	39
1	{D}	D	40
2	{B,C}	B	74
2	{B,C}	C	70
2	{B,D}	B	70
2	{B,D}	D	65
2	{C,D}	C	85
2	{C,D}	D	84
3	{B,C,D}	B	114
3	{B,C,D}	C	105
3	{B,C,D}	D	104

The final optimal cost is obtained by adding the return distance to A to each full-set value: $114 + 35 = 149$ through B, $105 + 39 = 144$ through C, and $104 + 40 = 144$ through D. The minimum of these is 144, so the optimal tour cost is 144.

3.3 Optimal Tour Result

Tracing back through the table gives the optimal tour: A to C to B to D and back to A, with individual leg distances 39, 35, 30 and 40, which sum to a total optimal cost of 144. This is the exact, globally minimum-cost tour among all six possible distinct tours of the four cities.

4. Greedy Approach (Nearest Neighbor Heuristic)

The Nearest Neighbor heuristic builds a tour incrementally. Starting from a chosen city, it repeatedly moves to the closest unvisited city until every city has been visited, and then returns to the starting city. It never

reconsiders a choice once made, which is what makes it a greedy algorithm rather than an exact one.

4.1 Algorithm Steps

- Step 1: Start at city A and mark it as visited.
- Step 2: From the current city, examine the distances to all unvisited cities and move to the one with the smallest distance.
- Step 3: Mark the newly reached city as visited.
- Step 4: Repeat Steps 2 and 3 until every city has been visited.
- Step 5: Return from the last visited city back to the starting city A to complete the tour.

4.2 Step-by-Step Execution

Step	Current City	Distances to Unvisited Cities	Nearest Choice	Leg Cost
1	A	B=35, C=39, D=40	B	35
2	B	C=35, D=30	D	30
3	D	C=45	C	45
4	C	return to A	A	39

The Nearest Neighbor heuristic therefore produces the tour A to B to D to C and back to A, with leg distances 35, 30, 45 and 39, giving a total greedy tour cost of 149.

5. Comparison of Total Tour Cost and Computational Effort

Criterion	Dynamic Programming (Held-Karp)	Greedy (Nearest Neighbor)
Tour Found	A - C - B - D - A	A - B - D - C - A

Total Cost	144 (optimal)	149
Deviation from Optimal	0 percent	about 3.5 percent higher
Subproblems / Decisions Evaluated	12 stored subproblem values, plus 6 final comparisons	4 local nearest-city decisions
Tours Implicitly Considered	All 6 distinct tours, evaluated exactly once each	Only 1 tour is ever constructed
Guarantee of Optimality	Always guaranteed	Not guaranteed

For this small four-city instance, the Dynamic Programming method used more computational effort, in the form of stored subproblem values, but it guaranteed the true minimum cost of 144. The Greedy method used far fewer computational steps, only four local decisions, but it produced a tour that is about 3.5 percent more expensive than the optimal solution because it committed early to the locally cheapest edges without considering their effect on the rest of the tour.

6. Why Greedy Does Not Guarantee Optimality

The Nearest Neighbor heuristic makes decisions using only local information available at the current step, and it never revisits or revises an earlier decision. In this instance, starting at A, the heuristic chose B because it was the closest city, then chose D from B, and was then forced to take the expensive edge from D to C, and finally the moderately expensive return edge from C to A. The true optimal tour instead routes through C immediately after A, which is a locally more expensive first step, but it arranges the remaining edges so that the overall total is smaller. This illustrates the general weakness of greedy algorithms on TSP: a

locally optimal choice at one step can close off access to a much better global arrangement later, and the algorithm has no mechanism to detect or correct this because it never looks ahead or backtracks. There is no known bound that keeps the Nearest Neighbor tour within a fixed small percentage of optimal for every possible instance, and on some distance matrices its error can be very large, growing without bound as the number of cities increases.

7. Complexity Analysis

7.1 Dynamic Programming Complexity

The Held-Karp algorithm stores one value for every combination of a subset of cities and an ending city within that subset. Since there are n cities, excluding the fixed start city there are $n - 1$ remaining cities, giving 2 raised to the power of $n - 1$ possible subsets, and up to $n - 1$ choices of ending city within each subset. Computing each entry requires scanning the previous subset, an operation of size proportional to n . This gives an overall time complexity of $O(n^2 \times 2^{n-1})$ and a space complexity of $O(n \times 2^{n-1})$, since every subproblem value must be kept in memory to be reused. For the four-city example in this assignment, this is a very small and fast computation, but the exponential factor 2 to the power n grows extremely quickly as n increases.

Number of Cities (n)	Approx. Time Steps ($n^2 * 2^{n-1}$)	Approx. Memory Entries ($n * 2^{n-1}$)
4	256	64
10	102,400	10,240
15	7,372,800	491,520
20	419,430,400	20,971,520

25	20,971,520,000	838,860,800
----	----------------	-------------

This table shows why exact Dynamic Programming solutions to TSP are described as having exponential complexity. Doubling the number of cities from 10 to 20 does not double the work, it multiplies it by roughly four thousand times, because the number of subsets doubles with every single additional city.

7.2 Greedy Nearest Neighbor Complexity

The Nearest Neighbor heuristic, at each of the n steps, scans the remaining unvisited cities to find the closest one, which takes time proportional to n at each step, giving an overall time complexity of $O(n^2)$. Its space complexity is only $O(n)$, since it only needs to remember which cities have been visited and the current position. This is dramatically cheaper than the Dynamic Programming approach for any reasonably large number of cities, which is precisely why the heuristic is preferred in practice despite not guaranteeing the optimal answer.

8. Scalability Challenges

- Dynamic Programming becomes computationally infeasible for large n because both its time and memory requirements grow exponentially; instances beyond roughly 20 to 25 cities already require billions of operations and gigabytes of memory, and the approach is effectively unusable for problems with hundreds or thousands of cities.
- The Greedy Nearest Neighbor heuristic scales comfortably to very large instances, since its polynomial time and linear space requirements remain manageable even for thousands of cities, but

the quality of its solution can degrade unpredictably, and it offers no guarantee on how far its answer will be from optimal.

- In real-world logistics and routing problems, which routinely involve hundreds or thousands of locations, exact methods such as Held-Karp are rarely used directly; instead, practitioners rely on heuristics, metaheuristics such as simulated annealing or genetic algorithms, or approximation algorithms that offer provable bounds close to optimal in reasonable time.
- A common practical compromise is to use a fast heuristic such as Nearest Neighbor to construct an initial tour quickly, and then apply local search improvement techniques, such as 2-opt or 3-opt edge exchanges, to refine that tour toward a better solution without the exponential cost of an exact method.

9. Insights: Exact Methods versus Heuristic Methods

- Exact methods such as Dynamic Programming guarantee correctness and always return the true optimal tour, which makes them valuable for small instances, for verifying the quality of heuristic solutions, and for problems where the cost of a wrong answer is very high.
- Heuristic methods such as Nearest Neighbor sacrifice the guarantee of optimality in exchange for speed and scalability, and they remain useful and often necessary for large practical problems where exact computation is not feasible within available time or memory.
- The choice between an exact and a heuristic method is fundamentally a trade-off between solution quality and computational feasibility, and the right choice depends on the size

of the problem, the acceptable margin of error, and the computational resources and time available.

- For the four-city example solved in this assignment, both methods were computationally cheap, but the Greedy heuristic already produced a tour about 3.5 percent worse than optimal, which illustrates that even on very small instances, greedy choices can miss the true optimum, and this gap generally tends to widen on larger and more irregular problem instances.

10. Conclusion

This assignment solved a four-city Travelling Salesperson Problem instance using both the exact Held-Karp Dynamic Programming algorithm and the approximate Nearest Neighbor Greedy heuristic. The Dynamic Programming method correctly found the optimal tour A to C to B to D to A with a total cost of 144, while the Greedy method produced the tour A to B to D to C to A with a higher total cost of 149. The comparison confirms that Dynamic Programming guarantees optimal correctness at the price of exponential time and space complexity, $O(2^n)$ and $O(2^n)$ respectively, while the Greedy heuristic runs in a much cheaper $O(n^2)$ time and $O(n)$ space but offers no guarantee of finding the optimal solution. This trade-off between guaranteed correctness and computational scalability is central to the study of combinatorial optimization, and it explains why large-scale routing problems in practice are generally solved using heuristic and metaheuristic techniques rather than exact dynamic programming.

RUBRICS FOR CALCULATION

Comparative Analysis					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%				
Comparison & Differentiation	25%				
Critical Evaluation	25%				
Synthesis & Insight Generation	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____

Date: _____

Date: _____

Date: _____